Technische Universität Berlin
Fakultät V – Verkehrs- und Maschinensysteme
Institut für Strömungsmechanik und Technische Akustik
Fachgebiet Technische Akustik
Einsteinufer 25, Sekr. TA7 – 10587 Berlin

SOFTWARE REQUIREMENTS SPECIFICATION

for

playamb - Ambisonics Player

Version 1.0 – Final

Authors: Giovanni Zebele (Matr. Nr. 515580)
Kylan Klein Lenderink (Matr. Nr. 524971)
Peiwen Xie (Matr. Nr. 526361)
Leihan Zhang (Matr. Nr. 525373)

Technical advisor: Konstantin Jefferson Fontaine

Course: Projekt Python & Akustik, SoSe 2026
Lecturer: Ennes Sarradj
Document date: July 20, 2026

1 Introduction

1.1 Purpose

This document specifies the functional and nonfunctional requirements of the Ambisonics Player developed in the context of the course "Projekt Python & Akustik" offered by the Department of Technical Acoustics at Technische Universität Berlin. It defines the purpose and scope of the software, its interfaces, required behaviour, operating constraints, and verification criteria.

The document is intended for developers, stakeholders, testers, and users of the software. It describes the final Version 1.0 scope and distinguishes between mandatory functionality, desirable functionality, experimental extensions, and future work.

1.2 Document Conventions

The requirement terms in this document follow the meanings in Table 1.1.

Term	Description
SHALL	A mandatory requirement that must be satisfied by Version 1.0 of the software.
SHOULD	A desirable requirement that is expected to be satisfied when technically feasible. A documented limitation is acceptable if full compliance cannot be demonstrated.
MAY	An optional capability or extension that is not required for acceptance of Version 1.0.

Table 1.1: Requirement terminology used in this document.

1.3 Project Scope

Ambisonics is a widely used format for spatial audio because of its flexible scene representation and compatibility with head-related transfer functions (HRTFs). It is therefore suitable for binaural reproduction over headphones. Established approaches to binaural Ambisonics decoding include rendering with interposed virtual loudspeakers (Schörkhuber

et al., 2018) and direct processing in the spherical-harmonic domain (Zotter & Frank, 2012).

The goal of this project is to develop a Python-based Ambisonics Player for binaural listening experiments and spatial-audio prototyping. The software reads AmbiX WAV files using ACN channel ordering and SN3D normalisation and renders them as binaural stereo output using spherical-harmonic HRTF filters.

The Version 1.0 scope includes:

- selection and chunk-based loading of AmbiX WAV files;
- automatic detection and user selection of the Ambisonics playback order;
- binaural SH-HRTF decoding using a predefined or user-selected SOFA HRTF data set;
- basic playback controls, including play, pause, resume, stop, volume, loop, start offset, and seeking with a transport bar;
- a graphical user interface for file selection, decoder settings, playback, status information, and scene rotation;
- manual and simulated head-rotation modes; and
- experimental support for compatible hardware head trackers.

The graphical user interface and SOFA loading were optional in the original project description but were adopted as part of the Version 1.0 project scope. Hardware head tracking was also implemented and tested as an experimental extension. Robust drift compensation, orientation smoothing, latency validation, and broad hardware compatibility remain future work.

The software SHALL follow object-oriented and modular design principles. The file loader, HRTF processing, renderer, playback engine, graphical user interface, and head-tracking integration SHALL remain separate components so that the core functionality can be imported into other Python applications. The source code SHALL be managed in a Git repository and accompanied by user and developer documentation.

2 Overall Description

2.1 Product Perspective

The Ambisonics Player is designed primarily as a standalone desktop application for listening to Ambisonics files of different orders over headphones. Its core loader, HRTF, renderer, playback, and head tracking components SHALL also be usable independently of the graphical user interface and importable into other Python projects or experimental environments.

Its primary responsibility is loading, binaurally decoding, rotating, and playing existing Ambisonics material.

2.2 Product Functions

The principal product functions are:

- select, validate, and load AmbiX WAV files;
- detect or select the Ambisonics playback order;
- load a default or user-selected SOFA HRTF data set;
- select available headphone compensation filters;
- convert Ambisonics audio into binaural stereo in real time;
- provide streaming playback and transport controls;
- display signal, decoder, playback, and tracking information;
- apply manual or simulated scene rotation; and
- connect to a supported head-tracking device as an experimental feature.

2.3 Operating Environment

2.3.1 Technical Environment

The application SHALL run on a desktop or laptop computer with a supported Python installation, sufficient working memory, an audio output device, and headphones. A graphical desktop environment is required for the use of the provided Tkinter interface, but it is not required for importing the core processing modules.

2.3.2 Hardware Environment

- A stereo audio output device and headphones are required for binaural playback.
- Processing performance depends on the CPU, selected Ambisonics order, block size, HRTF preprocessing method, and whether dynamic rotation is enabled.
- The software SHALL support files up to seventh order. Real-time operation at higher orders is a performance target and is not guaranteed on every computer.
- Hardware head tracking requires a compatible tracking device and its required operating-system or MIDI interfaces.

2.3.3 Software Environment

The reference implementation is written in Python. It uses libraries including `numpy`, `scipy`, `soundfile`, `sounddevice`, `pyfar`, `spharpy`, and related spatial-audio packages. The graphical user interface uses Tkinter. Experimental hardware tracking uses PyHeadTracker and the corresponding device or MIDI backend.

Required and optional dependencies SHALL be listed in `requirements.txt`. Version 1.0 is primarily tested on Windows. The core modules SHOULD remain portable to other desktop operating systems supported by the required dependencies.

2.3.4 User Environment

The intended users are students, researchers, developers, and audio engineers with a basic understanding of Ambisonics and binaural reproduction. Normal GUI operation SHALL not require command-line interaction after installation. Selection of a suitable Ambisonics file, HRTF, playback order, and headphone filter remains the responsibility of the user.

2.4 Design and Implementation Constraints

2.4.1 Technical Constraints

- The primary input format SHALL be an AmbiX WAV file using ACN channel ordering and SN3D normalization.
- The renderer SHALL preserve the input sampling rate. Automatic sample rate conversion is outside the mandatory Version 1.0 scope.
- The renderer SHALL convert the sampling rate of the HRTF if it does not match the input sampling rate. Sampling rate conversion can lead to HRTF truncation on upsampling to preserve performance.
- Processing SHALL use chunk-based streaming rather than loading the complete Ambisonics file into memory.
- Binaural decoding SHALL use spherical-harmonic-domain HRTF filters and block-wise FFT convolution.
- Higher Ambisonics orders require more processing time and memory; therefore, achievable real-time performance depends on the host computer.
- Optional backends and hardware devices SHALL not be required for manual rotation or normal playback with the documented baseline configuration.

2.4.2 User Constraints

Users SHALL provide valid AmbiX material and SHALL use headphones for the intended binaural reproduction. Users of hardware tracking SHOULD perform zero calibration while facing the intended reference direction. Because hardware tracking is experimental, users should be informed that drift and audible artefacts may occur during rapid orientation updates.

2.5 User Documentation

The project SHALL provide documentation that allows a user to install, configure, start, and operate the software without additional verbal instructions. The documentation SHALL include:

- supported Python versions and operating systems;
- dependency installation using `requirements.txt`;
- instructions for starting the graphical user interface;

- supported input formats and Ambisonics conventions;
- use of playback and decoder controls;
- HRTF and headphone filter selection;
- setup, detection, and calibration of supported head-tracking hardware;
- use of manual rotation and simulated tracking;
- troubleshooting information; and
- known limitations of the current version.

Public classes and functions in the core modules SHOULD contain docstrings describing their purpose, inputs, outputs, and relevant exceptions.

3 System Features

3.1 Description and Priority

3.1.1 Ambisonics File Loading

The software **SHALL** load AmbiX WAV files using ACN channel ordering and SN3D normalization. It SHALL validate the WAV container and channel count and obtain the file path, sampling rate, duration, frame count, and channel count.

If no order is selected, the loader **SHALL** determine the highest complete Ambisonics order supported by the input, up to seventh order. A user **SHALL** be able to select a lower playback order. If the requested order is too high, the system **SHALL** use the highest supported order and notify the user.

The loader **SHALL** read the input in chunks and support seeking without loading the complete file into memory.

3.1.2 HRTF and Binaural Rendering

The software **SHALL** provide a default HRTF and **SHOULD** load compatible custom HRTF data in SOFA format. Compatible headphone compensation filters **SHOULD** be selectable when available. The software **SHALL** default to using Diffuse Field Equalization as headphone filtering: Each individual HRTF is divided by the average of all HRTFs.

$$H_{\mathrm{DF},i}(f) = \mathrm{minphase}\left\{ \frac{H_i(f)}{\frac{1}{M}\sum_{j=1}^{M} |H_j(f)|^2 + \lambda(f)} \right\}$$

where M is the number of measured HRTF directions and $\lambda(f)$ is a frequency-dependent regularization term.

The renderer **SHALL** convert the selected Ambisonics channels into two-channel binaural output using spherical-harmonic HRTF filters. Implementing Spherical Harmonic functionality **SHALL** make use of the `spharpy` library. For each frequency bin f, the HRTF

measurements across all M directions are projected onto the spherical harmonic basis to obtain the coefficients:

$$\mathbf{h}_{nm}(f) = \mathbf{Y}^{\dagger}\,\mathbf{h}(f) = [h_{0,0},\ h_{1,-1},\ h_{1,0},\ h_{1,1},\ h_{2,-2},\ \ldots,\ h_{N,N}]^{\mathrm{T}}$$

where $\mathbf{Y}$ is an $M \times (N+1)^2$ matrix of real spherical harmonics evaluated at the measurement grid, and N is the maximum SH order. Once $\mathbf{h}_{nm}(f)$ is computed, it can be used to interpolate the HRTF at any target source position $(\hat{\theta}, \hat{\phi})$ by means of the inverse transform:

$$\hat{\mathbf{h}}(f) = \hat{\mathbf{Y}}\,\mathbf{h}_{nm}(f)$$

where $\hat{\mathbf{Y}}$ is the SH matrix evaluated at the new direction (pyfar Developers, n.d.).

Least-squares preprocessing **SHALL** be available as the baseline method of the renderer. Magnitude Least Squares (MagLS) **SHOULD** be used as the default preprocessing method. MagLS minimizes the magnitude error instead of the complex error, such that,

$$\mathbf{w}_{\mathrm{MagLS}} = \arg\min_{\mathbf{w}} \| |\mathbf{Y}\mathbf{w}| - |\mathbf{h}| \|^2$$

offering improved perceptual quality at high frequencies. (Engel et al., 2021b).

During playback, the renderer **SHALL** use block-wise FFT convolution, inverse FFT processing, and overlap-add reconstruction. Block sizes of 128, 256, 512, 1024, 2048, and 4096 samples **SHOULD** be supported. Real-time performance up to seventh order depends on the host computer and selected configuration.

3.1.3 Playback and Transport

The player **SHALL** provide play, pause, resume, stop, volume adjustment, loop playback, start offset, and progress-bar seeking. Stop **SHALL** reset the position to the beginning, while pause **SHALL** preserve the current position.

The GUI **SHALL** accept the start offset in seconds. The playback module **SHALL** validate the value and convert it internally to a sample index. Playback **SHALL** use streaming audio blocks and report the current position and playback state.

3.1.4 Graphical User Interface

The GUI **SHALL** provide file selection, transport controls, decoder settings, and clear loading, playback, tracking, and error states. It **SHALL** display the active source, sampling rate, channel count, duration, order, block size, HRTF, headphone filter, volume, loop state, and playback position.

Editable decoder settings **SHALL** be distinguishable from the configuration that is currently active. Decoder construction **SHOULD** run outside the GUI event thread. The GUI **SHALL** use the public interfaces of the core modules and **SHALL** not implement audio processing itself.

3.1.5 Rotation and Head Tracking

The software **SHALL** provide manual scene rotation and deterministic simulated tracking without external hardware.

Version 1.0 **SHOULD** connect to the supported reference tracker through PyHeadTracker. Hardware mode **SHOULD** support connection, start, stop, zero calibration, and yaw, pitch, and roll updates. Tracking data **SHALL** be processed outside the low-level audio callback.

If no compatible device is available, normal playback, manual rotation, and simulated tracking **SHALL** remain usable. Drift compensation, smoothing, latency guarantees, and broader device support are outside the mandatory Version 1.0 scope.

The software **MAY** allow the connection of OSC-compatible devices as sources of rotation data for hardware head tracking. It **MAY** allow the user to set a custom OSC message to parse the rotation values.

3.2 Functional Requirements

FR-L1: The loader **SHALL** load and validate AmbiX WAV files in ACN/SN3D format.

FR-L2: The loader **SHALL** detect or validate the selected order and handle unsupported requested orders with a clear warning.

FR-L3: The loader **SHALL** support chunk-based reading and seeking.

FR-R1: The renderer **SHALL** decode Ambisonics audio into binaural stereo using SH-HRTF processing.

FR-R2: The renderer **SHALL** provide the default HRTF and **SHOULD** load compatible custom SOFA files.

FR-R3: The renderer **SHALL** provide LS preprocessing and **SHOULD** additionally provide MagLS.

FR-R4: The renderer **SHALL** use block-wise FFT convolution and overlap-add reconstruction.

FR-P1: The player **SHALL** provide play, pause, resume, stop, volume, loop, start offset, and seek operations.

FR-P2: The player **SHALL** provide streaming playback and release its resources when closed.

FR-G1: The GUI **SHALL** provide file, playback, decoder, and rotation controls.

FR-G2: The GUI **SHALL** display the active signal, decoder, playback, and tracking state.

FR-G3: The GUI **SHALL** allow the user to switch between rotation data source (manual/simulated/hardware). Upon switching sources the current rotation values **SHALL** be discarded and reset to zero.
Only control buttons that are relevant to the selected source **SHOULD** be clickable. In detail:

- **a.** the "start Tracking" button **SHOULD** only be active when the tracking mode is set on "Demo" or "Hardware", otherwise deactivated. Upon starting tracking its name and function **SHOULD** become "Stop Tracking" and the other way around;
- **b.** the "Zero Tracker" button **SHOULD** only be clickable when the tracking mode is set on "Hardware", otherwise deactivated;
- **c.** Upon selection of a different tracking mode, the new tracking mode **SHOULD** be set and the GUI updated without needing any further interaction with the GUI.

FR-G4: The GUI **SHOULD** provide feedback on the current head orientation via a three-dimensional visualiser.

FR-H1: Manual and simulated rotation **SHALL** work without tracking hardware.

FR-H2: The system **SHOULD** connect to and read the supported reference head tracker (Supperware Head Tracker 1).

FR-H3: The system **MAY** connect to and read head orientation data from hardware devices that support communication via the Open Sound Control (OSC) protocol.

FR-H4: When connecting to head tracking devices via OSC, a custom OSC message **MAY** be provided for rotation data parsing.

FR-M1: Core loader, renderer, and playback functionality **SHALL** be usable independently of the GUI through documented Python interfaces.

FR-D1: The project **SHALL** document installation, operation, optional hardware, troubleshooting, and known limitations.

4 Nonfunctional Requirements

4.1 Performance Requirements

NFR-P1: Real-time processing. On the reference computer, the average processing time for one block **SHOULD** remain below the playback duration of that block. At 48 kHz 512, 1024, 2048, and 4096 samples correspond to approximately 10.7 ms 21.3 ms, 42.7 ms, and 85.3 ms.

NFR-P2: Memory use. During streaming playback, memory use **SHOULD** remain approximately independent of the total duration of the input file.

NFR-P3: Responsiveness. Loading and decoder construction **SHOULD** not freeze the GUI.

NFR-P4: Higher orders. The application **SHALL** accept complete AmbiX channel sets up to seventh order. Achievable real-time performance SHALL be documented for the reference computer.

4.2 Robustness Requirements

NFR-R1: Input validation. Invalid files, channel counts, orders, positions, or HRTF configurations **SHALL** produce a clear error or warning without corrupting the active playback state.

NFR-R2: Optional features. Missing optional hardware or processing methods **SHALL NOT** prevent use of the documented baseline playback configuration.

NFR-R3: Resource cleanup. The application **SHALL** close files, audio streams, device connections, and worker threads when replaced or when the application exits.

4.3 Software Quality Requirements

- File loading, HRTF processing, rendering, playback, GUI, and tracking **SHALL** remain separate modules.
- Mathematical helpers and loader validation **SHOULD** be independently testable.

- Required and optional dependencies **SHALL** be documented in `requirements.txt`.
- The GUI **SHALL** report the currently active configuration and **SHOULD** remain usable on smaller displays through vertical scrolling.
- Third-party code, algorithms, HRTF data, and filter resources **SHALL** retain required licensing and attribution.

5 Requirements Verification

5.1 Verification Strategy

Mandatory requirements SHALL be checked by inspection, automated tests, integration tests, performance measurements, or documented manual tests. The test configuration and result SHOULD be recorded.

Table 5.1 defines concise acceptance checks for the main functional and performance requirements. Requirement priorities are defined in Section 3.2 and are therefore not repeated in the table.

5.2 Reference Tests

Tests SHOULD record the operating system, processor, input order, sampling rate, block size, HRTF, headphone filter, preprocessing method, and tracking mode.

Hardware tracking SHOULD be tested by detecting and zeroing the device, checking yaw, pitch, and roll updates, confirming that the orientation reaches the audio rotation module, and releasing the connection. Observed drift, discontinuities, underflows, or audible artefacts SHOULD be recorded as limitations.

ID	Acceptance check
FR-L1	Load known ACN/SN3D AmbiX WAV files and verify their file metadata.
FR-L2	Compare detected and selected playback orders with known channel counts, including an unsupported requested order.
FR-L3	Play a large input file, monitor memory use, and seek to known positions.
FR-R1	Confirm two-channel binaural output and perform a documented listening test.
FR-R2a	Construct a decoder using the documented default HRTF.
FR-R2b	Construct a decoder using at least one compatible custom SOFA file.
FR-R3a	Construct and run the decoder using LS preprocessing.
FR-R3b	Run MagLS preprocessing when implemented.
FR-R4	Compare a short block-wise convolution result with an offline reference.
FR-P1	Test every specified transport operation in the GUI.
FR-P2	Run streaming playback and confirm that playback resources are released when the player closes.
FR-G1–G2	Operate the GUI and compare its displayed state with the active source, decoder, player, and tracking.
FR-G3	Start the GUI and change the tracking mode using the provided drop-down menu. Check that all the rotation/tracking controls have the desired behaviour.
FR-G4	Compare the visualised orientation and the expected directivity of the sound using test sound files. Connect a head tracker and compare the actual head movement with the displayed head rotation to prove consistency.
FR-H1	Run manual and simulated rotation without connected hardware.
FR-H2	Detect, open, zero, read, stop, and close the reference tracker.
FR-H3	Open, zero, read, stop, and close an OSC-compatible tracker.
FR-H4	Set a custom OSC message on both the sender and the tracker instance and check that the parsing of rotation data still works correctly.
FR-M1	Import and instantiate the core modules without starting the GUI.
FR-D1	Follow the installation and operating documentation in a clean environment.
NFR-P1	Measure processing time for representative orders and block sizes and compare it with the corresponding block duration.

Table 5.1: Requirements verification matrix.

6 Known Limitations and Future Work

6.1 Known Limitations

Version 1.0 has the following known limitations:

- Real-time performance at higher orders depends on the host computer, block size, HRTF processing method, and dynamic rotation state.
- Experimental hardware tracking may exhibit orientation drift and does not yet provide complete software smoothing or outlier rejection.
- Rapid rotation updates may produce audible discontinuities in some configurations.
- Output level may vary between input signals, Ambisonics orders, and decoder configurations. Version 1.0 does not provide perceptual loudness normalization.
- Hardware compatibility has been verified only with the reference device available to the development team.
- Automatic sample-rate conversion is outside the Version 1.0 scope.

6.2 Future Work

Future development should focus on orientation smoothing and drift compensation, crossfading between dynamically rotated filters, underflow and clipping monitoring, output-level normalization, additional tracking devices, and broader automated testing.

Bibliography

Engel, I., Goodman, D., & Picinali, L. (2021a). Assessing HRTF preprocessing methods for ambisonics rendering through perceptual models. *Acta Acustica*, *6*, 4. https://doi.org/10.1051/aacus/2021055

Engel, I., Goodman, D., & Picinali, L. (2021b). Improving binaural rendering with bilateral ambisonics and MagLS. *Tagungsband, DAGA 2021 - 47. Jahrestagung für Akustik*, 1608–1611. https://pub.dega-akustik.de/DAGA_2021/data/articles/000533.pdf

pyfar Developers. (n.d.). Spherical harmonic based HRTF interpolation [Accessed: 2026-07-20]. https://pyfar-gallery.readthedocs.io/en/latest/gallery/interactive/spherical_harmonic_hrtf_interpolation.html

Schörkhuber, C., Zaunschirm, M., & Höldrich, R. (2018). Binaural rendering of ambisonic signals via magnitude least squares. *Proceedings of the DAGA*, 339–342.

Zotter, F., & Frank, M. (2012). All-Round Ambisonic Panning and Decoding. *Journal of the Audio Engineering Society*, *60*(10), 807–820.